Assignment 1- NYT database exploration

Sofia Rodas

Exploring the appearance of the phrase “Sea Level Rise” in the New York Times

1. Create a free New York Times account (https://developer.nytimes.com/get-started).

2. Pick an environmental term of interest to you and use the {jsonlite} package to query the API. Pick something high profile enough and over a large enough time frame that your query yields enough articles for an interesting examination.

API Function

Define the preconstructed function that let’s us programatically call the API.

Note: The `Sys.sleep()` argument values were changed to trouble shoot issues with running the code on the desired phrase.

```
nytAPI = function(term1, begin_date, end_date, api) {
  searchQ = URLencode(term1)
  url = paste0('https://api.nytimes.com/svc/search/v2/articlesearch.json?q=',
searchQ,
               '&begin_date=', begin_date, '&end_date=', end_date, '&api-
key=', api)

  initialsearch = fromJSON(url, flatten = TRUE) # we create an initial search
to find out how many total hits, or articles that meet our query parameters
  hits = initialsearch$response$meta$hits

  if (hits == 0) return(data.frame()) ## if there are no hits, return an
empty data frame

  maxPages = min(round(hits / 10) - 1, 10) # otherwise, we find out how many
times we'll need to loop thru fcn, each time returning 10 pages

  df = data.frame(id=as.numeric(), created_time=character(),
                  snippet=character(), headline=character())

  for (i in 0:maxPages) {
    nytSearch = fromJSON(paste0(url, "&page=", i), flatten = TRUE)
    temp = data.frame(id = 1:nrow(nytSearch$response$docs),
                  created_time = nytSearch$response$docs$pub_date,
                  snippet = nytSearch$response$docs$snippet,
```

```
                    headline = nytSearch$response$docs$headline.main)
    df = rbind(df, temp)
    Sys.sleep(22)
  }
  return(df)
}
```

Connect to the New York Times API and send a query.

```
# define the term of interest
term1 <- c("sea level rise")

# save the data to a csv
results <- nytAPI(term1 = term1, begin_date = "20240401", end_date =
"20260401", api = API_KEY)

write.csv(results, paste0("NYT-", term1,".csv"))

# save the results from the API access for
nytDat <- results
```

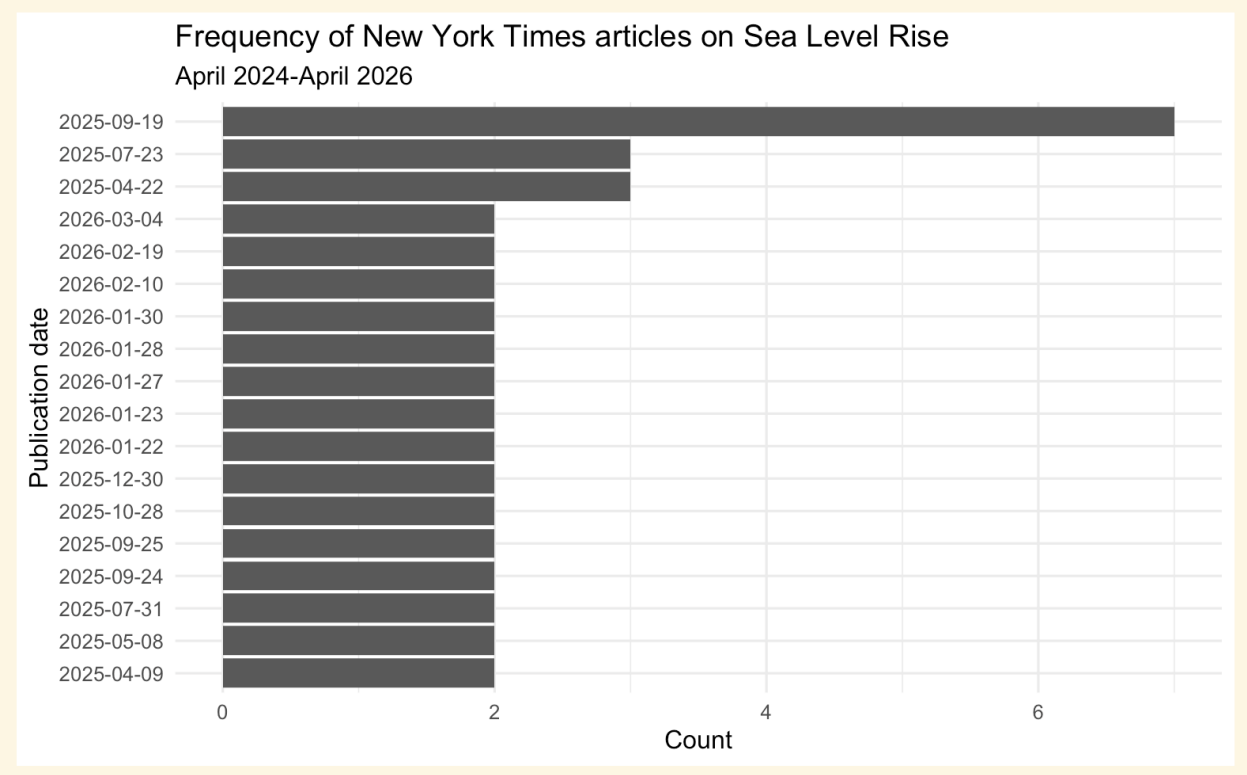
Note: For future use the saved file can be accessed. The sourcing of snippets and headers should be changed if the saved file is used. The snippets are changed from being sourced at 3 to 4 and headlines are changed from being sourced at 4 to 5.

3. Recreate the publications per day and word frequency plots using the snippets.

- Make some (at least 3) transformations to the corpus (add context-specific stopword(s), simplify possessives, remove numbers)

Visualize the number of publications discussing sea level rise and the dates of each publication.

```
publication_dates_graph <- nytDat |>
  mutate(pubDay = as.Date(created_time)) |>
  group_by(pubDay) |>
  summarise(count=n()) |>
  filter(count >= 2) |>
  ggplot() +
  geom_bar(aes(x=reorder(pubDay, count), y=count), stat="identity") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Frequency of New York Times articles on Sea Level Rise",
       subtitle = "April 2024-April 2026",
       y = "Count",
       x = "Publication date")

# show the publication dates graph
publication_dates_graph
```



Climate week NYC took place from September 19, 2025 to September 29, 2025. September 19th, 2025 had the most articles about sea level rise. September 24th and 25th 2025 were also listed as having sea level rise appear twice.

Tokenization of the snippets

The snippets are stored in the 3rd column of the dataframe. Here we reassign the snippets and tokenize them.

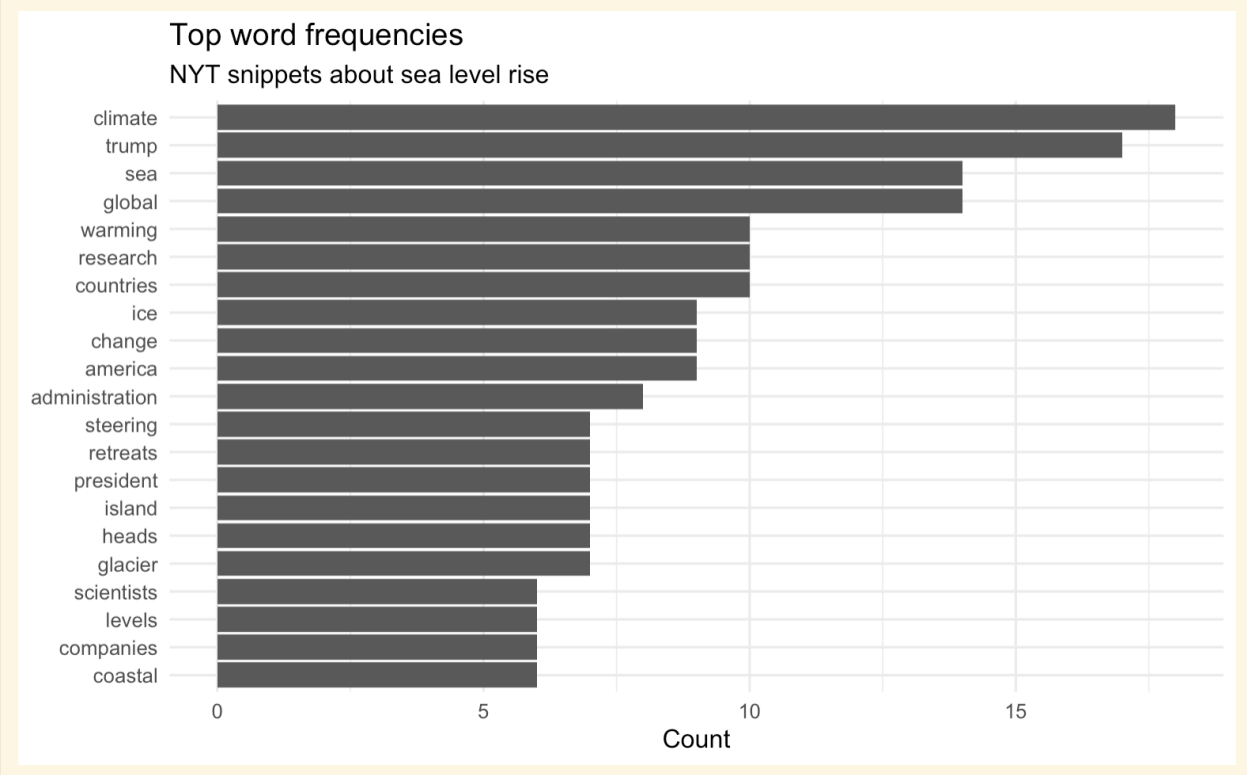
```
# save the snippets from the accessed data
snippet <-  names(nytDat)[3]

# use tidytext::unnest_tokens to put in tidy form
tokenized <- nytDat |>
  unnest_tokens(word, snippet)
```

All of the filler words are still present. Let's remove the filler words.

Clean the tokenized snippets

```
# access the stop words
data(stop_words)
stop_words

## # A tibble: 1,149 × 2
##    word        lexicon
##    <chr>       <chr>
```

```
##  1 a           SMART
##  2 a's         SMART
##  3 able        SMART
##  4 about       SMART
##  5 above       SMART
##  6 according   SMART
##  7 accordingly SMART
##  8 across      SMART
##  9 actually    SMART
## 10 after       SMART
## # ℹ 1,139 more rows

# remove stop words from snippets
tokenized <- tokenized |>
  anti_join(stop_words)

## Joining with `by = join_by(word)`

# remove numbers from snippets
clean_tokens <- str_remove_all(tokenized$word, "[:digit:]")

# remove possessives from snippets
clean_tokens <- gsub("’s", '', clean_tokens)

tokenized$clean <- clean_tokens

# drop where clean is empty
tokenized <- tokenized |> filter(clean != "")

# drop they're (the type of apostrophe stops it from being dropped with the
stop words)
tokenized <- tokenized |> filter(clean != "they’re")

# visualize the most used words
snippet_graph <- tokenized |>
  count(clean, sort = TRUE) |>
  filter(n > 5) |>
  mutate(clean = reorder(clean, n)) |>
  ggplot(aes(n, clean)) +
  geom_col() +
  labs(title = "Top word frequencies",
       subtitle = "NYT snippets about sea level rise ",
       y = NULL,
       x = "Count") +
  theme_minimal()

# show the snippet word count graph
snippet_graph
```



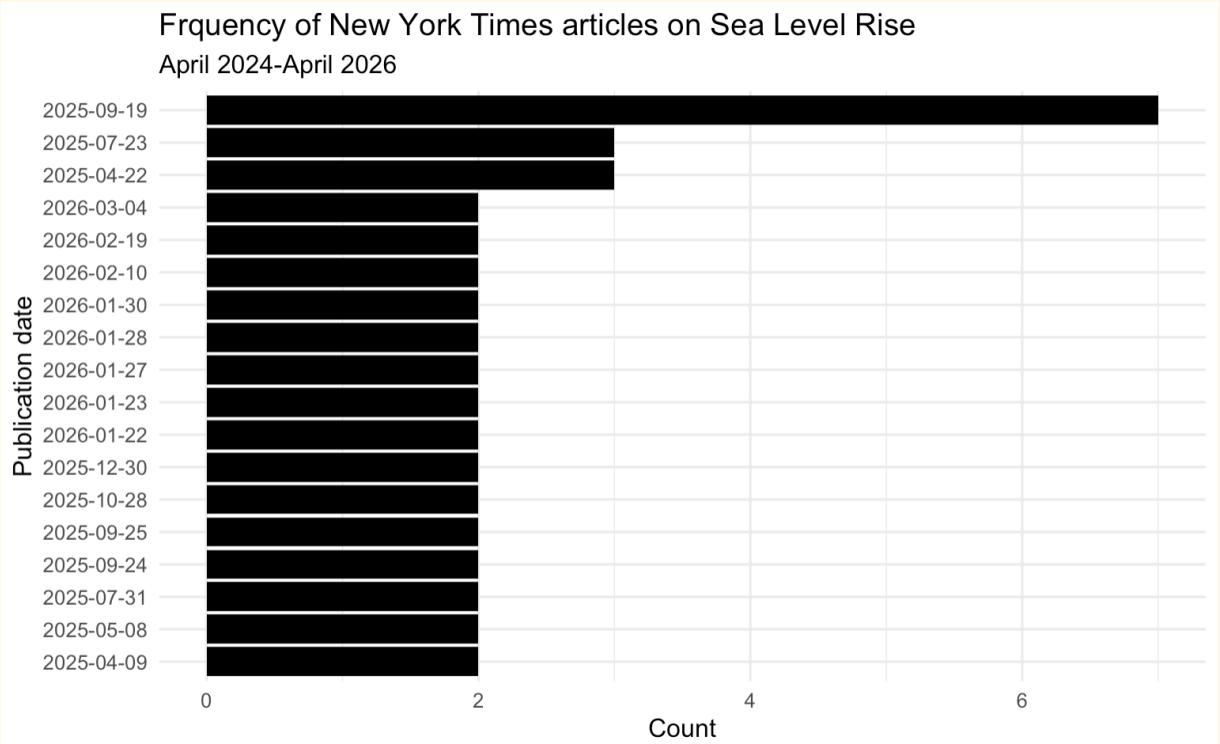
4. Recreate the publications per day and word frequency plots using the headlines variable (response.docs.headline.main). Compare the distributions of word frequencies between the snippet and headlines. Do you see any differences? Why might this be?

####Visualize the number of publications discussing sea level rise and the dates of each publication.

The API searches for the search term within on both headlines and snippets and retrieves the headlines and snippets for all of the hits. Therefore all of the dates are the same for both headlines and snippets. The publication date graphs are constructed using the exact same formatting.

```
publication_headline_graph <- nytDat |>
  mutate(pubDay = as.Date(created_time)) |>
  group_by(pubDay) |>
  summarise(count=n()) |>
  filter(count >= 2) |>
  ggplot() +
  geom_bar(aes(x=reorder(pubDay, count), y=count),
           stat="identity",
           fill = "black") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Frquency of New York Times articles on Sea Level Rise",
       subtitle = "April 2024-April 2026",
```

```
       y = "Count",
       x = "Publication date")

# show the headline date graph
publication_headline_graph
```



Tokenization of the headlines

The headlines are stored in the fourth column when accessing them from the API returned data.

```
# save the headlines from the accessed data
headlines <-  names(nytDat)[4]

# use tidytext::unnest_tokens to put in tidy form
headlines_tokenized <- nytDat |>
  unnest_tokens(word, headlines)
```

Clean the tokenized headlines

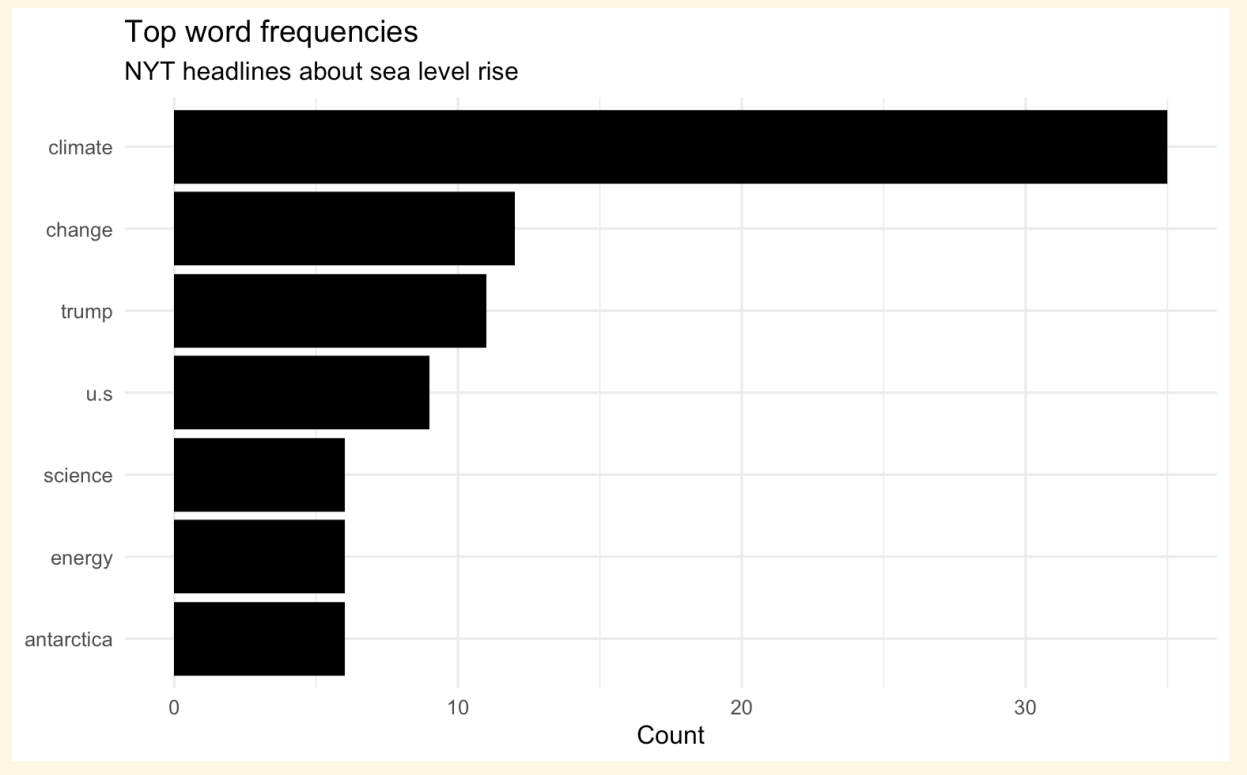
```
# access the stop words
data(stop_words)
stop_words

## # A tibble: 1,149 × 2
##    word        lexicon
##    <chr>       <chr>
```

```
##  1 a           SMART
##  2 a's         SMART
##  3 able        SMART
##  4 about       SMART
##  5 above       SMART
##  6 according   SMART
##  7 accordingly SMART
##  8 across      SMART
##  9 actually    SMART
## 10 after       SMART
## # ℹ 1,139 more rows
```

```
 # remove stop words
headlines_tokenized <- headlines_tokenized |>
  anti_join(stop_words)
```

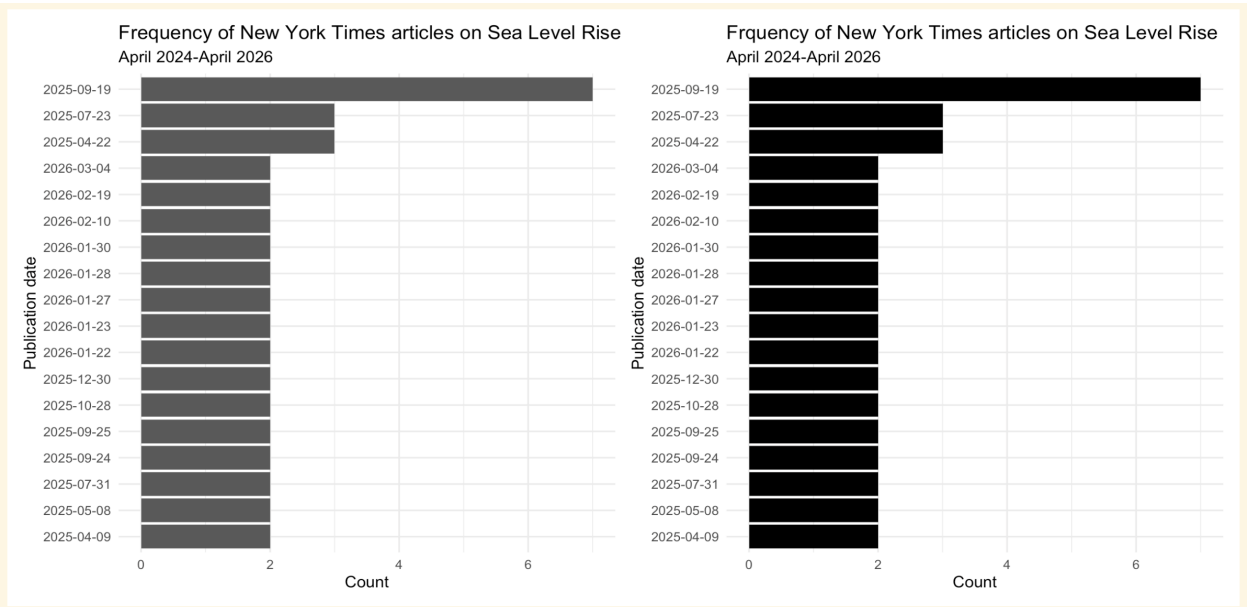
```
## Joining with `by = join_by(word)`
```

```
#remove all numbers
clean_headlines_tokens <- str_remove_all(headlines_tokenized$word,
"[:digit:]")

# remove possessives
clean_headlines_tokens <- gsub("’s", '', clean_headlines_tokens)

headlines_tokenized$clean <- clean_headlines_tokens

# drop where clean is empty
headlines_tokenized <- headlines_tokenized |> filter(clean != "")



# visualize the most used words for headlines
headlines_graph <- headlines_tokenized |>
  count(clean, sort = TRUE) |>
  filter(n > 5) |>
  mutate(clean = reorder(clean, n)) |>
  ggplot(aes(n, clean)) +
  geom_col(fill = "black") +
  labs(title = "Top word frequencies",
       subtitle = "NYT headlines about sea level rise",
       y = NULL,
       x = "Count") +
  theme_minimal()

# show the headlines graph
headlines_graph
```



Publication date comparison

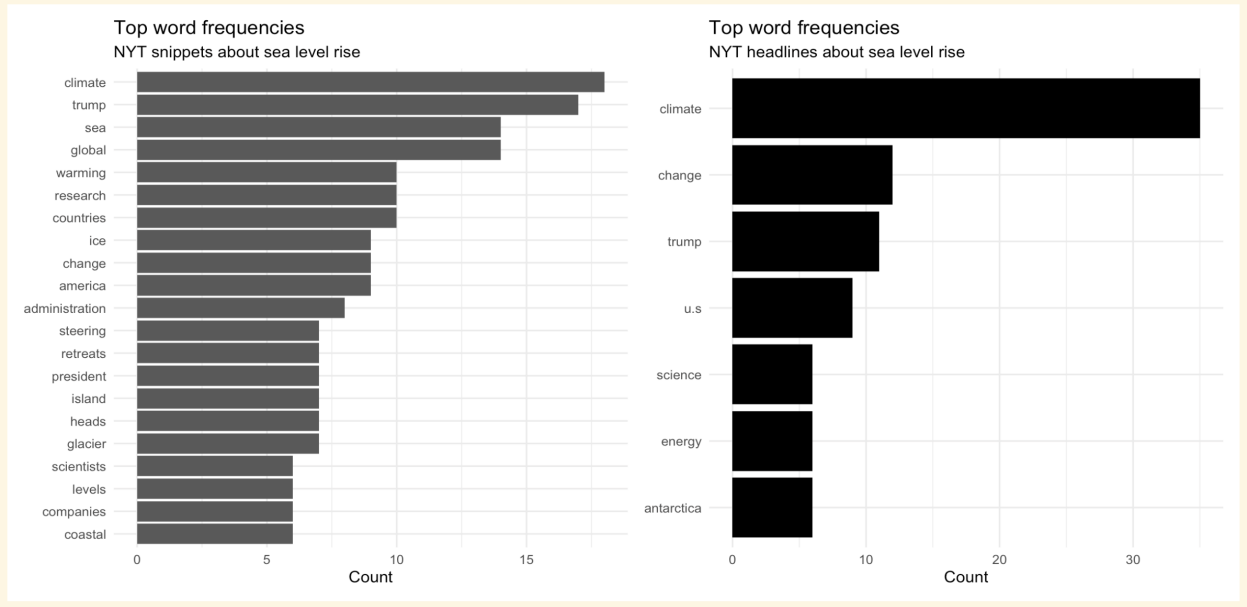
```
publication_dates_graph + publication_headline_graph
```



As described above, based on the way the function is written, the API searches for the search term in both the headlines and the snippets and returns both of the corpus and the publication dates. Hence the publication dates are the same whether looking at them for the snippets of the headlines.

Compare the graphs showing the frequency of words in snippets versus headlines

```
snippet_graph + headlines_graph
```



There are many more words with a count larger than 5 when analyzing snippets compared to the number of words with a count larger than 5 when analyzing headlines. This makes sense since snippets are more text than headlines. The words “climate”, “change”, and “trump” are found more than 5 times in both the snippets and the headlines. The remaining words still revolve around the topic of sea level rise. The words other than “climate”, “change”, and “trump” either appeared only more than 5 times in the snippets or the headlines.

Submit finished labs in both .Rmd and knitted PDF or html form to Canvas.

***Remember that complete assignments require readable, informative, aesthetically pleasing visualizations. You’ll have to improve on the bare bones versions in this demo.*